

Driving with Smart AI Steering Car

Jerell, Zhun Hang, Lance & Faliq

1 Introduction

The concept of self-driving cars has intrigued engineers and software developers since the 1920s. The journey towards autonomy began with the introduction of Advanced Driver Assistance Systems (ADAS), starting with Cruise Control in 1948. The first semi-autonomous car was unveiled in 1977 by Japan's Tsukuba Mechanical Engineering Laboratory. This vehicle navigated specially marked streets using two cameras and an analog computer to interpret the environment.

As substantial investments continue to flow into the research and development of autonomous vehicles in the physical world, driving AI in game development has also seen significant advancements, particularly with the integration of deep learning AI into video games. Early implementations featured cars that followed predefined paths, with game developers drawing racing lines and the AI cars applying throttle and brakes at specified points to achieve the ideal lap designed by the developers.

Despite these advancements, there remain limitations in the driving AI used in simulation-type games such as Cities: Skylines and Grand Theft Auto V (GTA V). The AI in these games often follows a set of rules that sometimes do not align with real-world behavior.

We are researching the development and implementation of a basic driving AI that simulates real-world driving behavior in a 2D top-down game environment. Our focus is primarily on creating AI that can follow traffic rules, avoid obstacles, and adapt to dynamic changes in the environment, such as the presence of other vehicles and pedestrians.

Smart AI Steering Car is a car simulator designed to showcase a vehicle navigating down a street autonomously. Its core features include automatic lane changing, adherence to traffic light signals, and stopping for pedestrian crossings.

2 Basics of Driving

Driving is a complex task that involves following a set of rules and responding to dynamic environmental conditions. For a self-driving car, understanding and adhering to these rules is crucial to ensure safety and efficiency. In our 2D top-down game environment, we aim to simulate real-world driving behavior by implementing several fundamental driving rules and principles.

2.1 Traffic Rules

The core any driving AI simulator is its ability to follow traffic rules. In our demo, we focus on the following key rules

2.1.1 Lane Changing

Lane changing is the process of moving from one lane to another, typically to avoid obstacles or overtake slower vehicles. In our demo, the car will constantly scan the obstacles ahead of the car in the current lane, as well as the neighbouring lanes. When an obstacle is detected, the AI will evaluate the adjacent lanes for availability and safety before executing a lane change maneuver.

2.1.2 Traffic Light Signals

A traffic light is included in the simulator which will help control the flow of traffic at intersections by indicating to vehicles when they should stop, proceed or slow down. In our demo, we have programmed the vehicle to recognise the traffic light signals. It will stop at red lights, proceed on green lights, and slow down on yellow lights.

2.1.3 Pedestrian Crossings

Pedestrian crossings are designated area where pedestrians have the right of way to cross the street. As such, vehicles are required to stop to allow for pedestrians to cross the road. In our demo, the AI will detect pedestrian crossings and comes to a complete stop when pedestrians are present, allowing them to cross safely before proceeding.

3 Implementation

In this section, we will discuss the technical aspect of how we implemented the key features of our demonstration. This include the methodology, algorithm and any Unity-specific components used to achieve the key features.

3.1 Lane Change

3.1.1 Lane Detection

```
#region Switch Lanes
private void CheckForIncomingCars()
{
    /* Stop Checking incoming cars when switching lanes */
    if (lane.isSwitchingLane) return;

    /* Center Positions for lane checking */
    Vector2 currentLanePosition = (Vector2)transform.position + Vector2.up * detectionBoxOffsetY;
    Vector2 leftLanePosition = currentLanePosition + Vector2.left * 5f;
    Vector2 rightLanePosition = currentLanePosition + Vector2.right * 5f;

    /* Current Lane incoming cars */
    Collider2D[] incomingCarsCurrentLane = Physics2D.OverlapBoxAll(currentLanePosition, detectionBoxSize, 0f, carLayerMask);

    foreach (Collider2D collider in incomingCarsCurrentLane)
    {
        if (collider != GetComponent<Collider2D>())
        {
            lane.isCurrentLaneClear = false;
            break;
        }
    }

    /* Lane to the right of Main Car incoming cars */
    if (lane.currentLane == Lane.SelectedLane.Left || lane.currentLane == Lane.SelectedLane.Middle)
    {
        Collider2D[] incomingCarsRight = Physics2D.OverlapBoxAll(rightLanePosition, detectionBoxSize, 0f, carLayerMask);

        foreach (Collider2D collider in incomingCarsRight)
        {
            if (collider != GetComponent<Collider2D>())
            {
                lane.isRightClear = false;
                break;
            }
        }
    }

    /* Lane to the left of Main Car incoming cars */
    if (lane.currentLane == Lane.SelectedLane.Right || lane.currentLane == Lane.SelectedLane.Middle)
    {
        Collider2D[] incomingCarsLeft = Physics2D.OverlapBoxAll(leftLanePosition, detectionBoxSize, 0f, carLayerMask);

        foreach (Collider2D collider in incomingCarsLeft)
        {
            if (collider != GetComponent<Collider2D>())
            {
                lane.isLeftClear = false;
                break;
            }
        }
    }
}
```

```

/* Switch Lanes based on flags */
if (!lane.isCurrentLaneClear && trafficLight.CurrentLight != TrafficLight.Light.RED)
{
    if (lane.currentLane == Lane.SelectedLane.Middle)
    {
        if (!lane.isLeftClear && lane.isRightClear)
        {
            StartCoroutine(SwitchLaneCoroutine(Lane.SelectedLane.Right));
        }
        else if (!lane.isRightClear && lane.isLeftClear)
        {
            StartCoroutine(SwitchLaneCoroutine(Lane.SelectedLane.Left));
        }
        else
        {
            StartCoroutine(SwitchLaneCoroutine(Lane.SelectedLane.Left));
        }
    }
    else if (lane.currentLane == Lane.SelectedLane.Left && lane.isRightClear)
    {
        StartCoroutine(SwitchLaneCoroutine(Lane.SelectedLane.Middle));
    }
    else if (lane.currentLane == Lane.SelectedLane.Right && lane.isLeftClear)
    {
        StartCoroutine(SwitchLaneCoroutine(Lane.SelectedLane.Middle));
    }
}

/* Reset Lane flags after checks */
lane.isLeftClear = true;
lane.isRightClear = true;
lane.isCurrentLaneClear = true;
}

```

3.1.2 Execution of Lane Change

```

private IEnumerator SwitchLaneCoroutine(Lane.SelectedLane targetLane)
{
    lane.isSwitchingLane = true;
    Vector3 startPosition = transform.position;
    Vector3 endPosition = transform.position;

    /* Calculate end position */
    if (targetLane == Lane.SelectedLane.Left)
        endPosition.x -= 5f;
    else if (targetLane == Lane.SelectedLane.Right)
        endPosition.x += 5f;
    else if (targetLane == Lane.SelectedLane.Middle)
        endPosition.x = 0f;

    /* Smooth interpolation of lane switches */
    float elapsedTime = 0f;
    while (elapsedTime < laneSwitchDuration)
    {
        transform.position = new Vector3(Mathf.Lerp(startPosition.x, endPosition.x, elapsedTime
            / laneSwitchDuration), transform.position.y, transform.position.z);
        elapsedTime += Time.deltaTime;
        yield return null;
    }

    transform.position = new Vector3(endPosition.x, transform.position.y, transform.position.z);
    lane.currentLane = targetLane;
    lane.isSwitchingLane = false;
}

```

3.2 Traffic Light Signals

3.2.1 Traffic Light Recognition

```
private void RespondToLight()
{
    switch (traffic.CurrentLight)
    {
        case TrafficLight.Light.RED:
            carSpeed.speed = 0f;
            break;
        case TrafficLight.Light.AMBER:
            if (!isSlowingDown)
            {
                StartCoroutine(SlowDown());
            }
            break;
        case TrafficLight.Light.GREEN:
            if (!isSpeedingUp)
            {
                StartCoroutine(SpeedUp());
            }
            break;
        default:
            break;
    }
}
```

3.2.2 Behavior at Traffic Lights

```
private IEnumerator SlowDown()
{
    isSlowingDown = true;
    float elapsedTime = 0f;
    float slowDownDuration = 2f;
    float initialSpeed = carSpeed.speed;
    float targetSpeed = 0f;

    while (elapsedTime < slowDownDuration)
    {
        carSpeed.speed = Mathf.Lerp(initialSpeed, targetSpeed, elapsedTime / slowDownDuration);
        elapsedTime += Time.deltaTime;
        yield return null;
    }

    carSpeed.speed = targetSpeed;
    isSlowingDown = false;
}

private IEnumerator SpeedUp()
{
    isSpeedingUp = true;
    float elapsedTime = 0f;
    float speedUpDuration = 2f;
    float initialSpeed = carSpeed.speed;
    float targetSpeed = carSpeed.originalSpeed;

    while (elapsedTime < speedUpDuration)
    {
        carSpeed.speed = Mathf.Lerp(initialSpeed, targetSpeed, elapsedTime / speedUpDuration);
        elapsedTime += Time.deltaTime;
        yield return null;
    }

    carSpeed.speed = targetSpeed;
    isSpeedingUp = false;
}
```

3.3 Pedestrian Crossings

3.3.1 Pedestrian Detection

```
#region Pedestrian Crossing
private void PedestrianCrossing()
{
    /* Check if Zebra Crossing is in range */
    if (zebraCrossing.IsInCamera() && Mathf.Abs(zebraCrossing.gameObject.transform.position.y
        - gameObject.transform.position.y) <= 10.0f)
    {
        Debug.Log("Zebra crossing in range.");

        /* Locate any Pedestrian that are crossing the Zebra Crossing */
        pedestrian = GameObject.Find("Pedestrian");

        if (pedestrian != null)
        {
            Debug.Log("Pedestrian detected.");
            pedestrianCrossing = pedestrian.GetComponent<PedestrianCrossing>();

            if (pedestrian.transform.position.x == 10f)
            {
                /* Check if Pedestrian is crossing */
                if (!pedestrianCrossing.isCrossing && !PedestrianIsCrossing)
                {
                    /* Start Slowing down the Car */
                    StartCoroutine(SlowDownCar());

                    /* Allow Pedestrian to cross */
                    pedestrianCrossing.goAheadAndCross = true;
                    PedestrianIsCrossing = true;
                }
            }
            /* Pedestrian done crossing */
            else if (pedestrian.transform.position.x == -10f)
            {
                ResumeDriving();
            }
        }
        else
        {
            Debug.Log("Pedestrian not found.");
        }
    }
}
```

3.3.2 Stopping for Pedestrians

```
private void ResumeDriving()
{
    if (pedestrianCrossing != null && !pedestrianCrossing.isCrossing)
    {
        PedestrianIsCrossing = false;
        StartCoroutine(SpeedUpCar());
    }
}

private IEnumerator SlowDownCar()
{
    float initialSpeed = carSpeed.speed;
    float elapsedTime = 0f;
    float slowDownDuration = 2f;

    Debug.Log("Slowing down car.");
    while (elapsedTime < slowDownDuration)
    {
        carSpeed.speed = Mathf.Lerp(initialSpeed, 0f, elapsedTime / slowDownDuration);
        Debug.Log($"Slowing down... current speed: {carSpeed.speed}");
        elapsedTime += Time.deltaTime;
        yield return null;
    }

    carSpeed.speed = 0f;
    Debug.Log("Car stopped for pedestrian.");
}

private IEnumerator SpeedUpCar()
{
    float initialSpeed = carSpeed.speed;
    float targetSpeed = carSpeed.originalSpeed;
    float elapsedTime = 0f;
    float speedUpDuration = 3f;

    Debug.Log("Speeding up car.");
    while (elapsedTime < speedUpDuration)
    {
        carSpeed.speed = Mathf.Lerp(initialSpeed, targetSpeed, elapsedTime / speedUpDuration);
        Debug.Log($"Speeding up... current speed: {carSpeed.speed}");
        elapsedTime += Time.deltaTime;
        yield return null;
    }

    carSpeed.speed = targetSpeed;
    Debug.Log("Car resumed driving.");
}
```

4 Summary and Conclusion

In this research paper, we explored the development and implementation of a basic driving AI system within a 2D top-down game environment. Our primary objective was to simulate real-world driving behavior by creating AI that adheres to traffic rules, avoids obstacles, and responds dynamically to changes in the environment.

4.1 Key Achievements

During the project's implementation, we developed AI capabilities for lane changing, traffic light adherence, pedestrian crossing stops. The AI demonstrated the ability to:

- Detect lane boundaries and execute safe lane changes.
- Recognize and react appropriately to traffic light signals.
- Detect pedestrians at crossings and stop to allow safe passage.

4.2 Challenges

Despite these successes, we encountered several challenges:

- **Traffic Light Timing:** Ensuring accurate timing and appropriate AI reactions to different traffic light signals.
- **Pedestrian Safety:** Implementing a reliable detection system for pedestrians to prevent accidents.

There are still some bugs in pedestrian detection. Moreover, there are times where the traffic light would stop vehicles that are past the traffic light as well. This issue highlights the need for further refinement and testing.

4.3 Future Work

While our project achieved its primary goals, there are several areas for future improvement and expansion:

- **Advanced Obstacle Detection and Avoidance:** Implementing more sophisticated techniques, such as machine learning-based vision systems.
- **Complex Traffic Scenarios:** Introducing multi-lane highways, roundabouts, and merging lanes to simulate more complex driving environments.
- **Realistic Vehicle Interactions:** Enhancing the AI to handle interactions with various vehicle types and behaviors in more realistic settings.

4.4 Conclusion

Our project successfully demonstrated the development of a basic driving AI system capable of following traffic rules and avoiding obstacles in a 2D top-down game environment. By leveraging Unity's powerful features and incorporating key driving principles, we created an AI system that mimics real-world driving behavior. This work lays the groundwork for further advancements in autonomous driving simulation, paving the way for more complex and realistic AI systems in the future.